

Notes on Reinforcement Learning

Hyoung Joon, Kim

June 12, 2026

Contents

1	Multi-armed Bandits	3
1.1	Introduction	3
1.2	k -armed Bandit Problem	3
1.3	Balancing between Exploring and Exploiting	3
1.3.1	Action-Value Methods	3
1.3.1.1	Estimation of Value	4
1.3.1.1.1	Methods for Stationary Bandit Problem	4
1.3.1.1.2	Methods for Non-stationary Bandit Problem	4
1.3.1.2	Selection of Actions	5
1.3.1.2.1	Greedy Action Selection	5
1.3.1.2.2	ϵ -greedy Selection	5
1.3.1.2.3	Upper-Confidence-Bound Action Selection	6
1.3.1.3	Optimistic Initial Values	6
1.3.2	Gradient Bandit Algorithm	7
1.3.2.1	Algorithm	7
2	Markov Decision Processes	9
2.1	Introduction	9
2.2	Finite Markov Decision Process	9
2.2.1	Definition	9
2.2.2	Markov Property	10
2.3	Episodic Task and Continuing Task	10
2.4	Policies and Value Functions	11
2.4.1	Definitions	11

2.4.2	Bellman Equation	12
2.5	Optimal Policies and Optimal Value Functions	13
2.5.1	Definition	13
2.5.2	Bellman Optimality Equation	13
3	Dynamic Programming	15
3.1	Introduction	15
3.2	Policy Evaluation	15
3.3	Policy Improvement	16
3.4	Obtaining Optimal Policy	17
3.4.1	Policy Iteration	17
3.4.2	Value Iteration	18
3.5	Asynchronous Dynammic Programming	19
3.6	Generalized Policy Iteration	19

Chapter 1

Multi-armed Bandits

1.1 Introduction

In this chapter, we discuss about the evaluative aspect of reinforcement learning in simplified setting. We consider situation with only one *state*, with multiple choices. This can be understood as k -armed bandit problem.

1.2 k -armed Bandit Problem

In k -armed bandit problems, we can pick one of the k actions at time step t , and receive the reward and then determine the action of next time step $t + 1$, using the reward. Our objective is to maximize the total expected reward over some time period. Following are some notations and definitions:

- Action selected on time step t : A_t ;
- Reward given on time step t : R_t ;
- The expected reward of selected action a (called value): $q_*(a) = \mathbb{E}[R_t | A_t = a]$

If the values of actions are known exactly, we just have to select the action which has largest value at each time step. So we assume that the exact values of actions are not known, and this is the usual situation, too. Since we do not know exact values, we try to estimate the values.

1.3 Balancing between Exploring and Exploiting

1.3.1 Action-Value Methods

Since we assume that the exact value of each actions are not known, we try to estimate the value of actions, and use it to make action selection decisions. This is collectively called as **action-value methods**.

1.3.1.1 Estimation of Value

1.3.1.1.1 Methods for Stationary Bandit Problem The reward probabilities does not change over time in *stationary problems*. The following is a natural way of estimating the value of actions.

Definition 1.3.1.1 (Sample-average Method)

Let A_i and R_i be the action selected at time step i and the corresponding reward. Then, the **sample-average** method estimates the value of action a at time t as following:

$$Q_t(a) = \frac{\text{sum of rewards when } a \text{ taken prior to } t}{\text{number of times } a \text{ taken prior to } t} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbb{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbb{1}_{A_i=a}}$$

It is guaranteed to converge to true value, $q_*(a)$, as the time step increases. This can be verified with the law of large numbers.

Considering computational efficiencies, recording all the rewards of each actions selected each time is not memory-efficient. We can use incremental method to implement the sample-average method efficiently.

Algorithm 1: Incremental Sample Average Algorithm

Input : t current time step
 a selected action in time step t
 n the times of action a was selected until time step t (excluding current selection)
 R_n reward given to n -th selected action a
 $Q_t(a)$ estimation of value of action a in current time step t
Output: $Q_{t+1}(a)$ new estimate of value of action a

```
1  $Q_{t+1}(a) \leftarrow Q_t(a) + \frac{1}{n}(R_n - Q_t(a))$ 
2 return  $Q_{t+1}(a)$ 
```

1.3.1.1.2 Methods for Non-stationary Bandit Problem The averaging methods discussed above are not appropriate for *non-stationary* bandit problem, whose reward probabilities changes over time. For non-stationary bandit problem, we give more weight to recent rewards of action than to long-past rewards. We can implement this via using the moving average.

Algorithm 2: Weighted Average Method Algorithm

Input : t current time step
 a selected action in time step t
 n the times of action a was selected until time step t (excluding current selection)
 R_n reward given to n -th selected action a
 $Q_t(a)$ estimation of value of action a in current time step t
 $\alpha \in (0, 1]$ constant step size
Output: $Q_{t+1}(a)$ new estimate of value of action a

```
1  $Q_{t+1}(a) \leftarrow Q_t(a) + \alpha(R_n - Q_t(a))$ 
2 return  $Q_{t+1}(a)$ 
```

We can verify that this algorithm weakens the effect of rewards given earlier. We first assume that $Q_n(a)$ is the value of

action a at the time step when the action a is selected for n times.

$$\begin{aligned}
Q_{n+1}(a) &= Q_n(a) + \alpha(R_n - Q_n(a)) \\
&= \alpha R_n + (1 - \alpha)Q_n(a) \\
&= \alpha R_n + (1 - \alpha)(\alpha R_{n-1} + (1 - \alpha)Q_{n-1}(a)) = \alpha R_n + (1 - \alpha)\alpha R_{n-1} + (1 - \alpha)^2 Q_{n-1}(a) \\
&= \alpha R_n + (1 - \alpha)\alpha R_{n-1} + (1 - \alpha)^2 \alpha R_{n-2} + \dots + (1 - \alpha)^{n-1} \alpha R_1 + (1 - \alpha)^n Q_1(a) \\
&= (1 - \alpha)^n Q_1(a) + \sum_{i=1}^n \alpha (1 - \alpha)^{n-i} R_i
\end{aligned}$$

As one can see, the earlier rewards decreases exponentially.

The step size α can vary from step to step. Then, the conditions required for $Q_t(a)$ to converge to true action value with probability 1 is given as:

Theorem 1.3.1.1 (Conditions for Convergence)

Let $\alpha_n(a)$ be the step-size parameter used to process the reward received after the n -th selection of action a . Then, $Q_t(a)$ converges to $q_*(a)$ as $t \rightarrow \infty$ with probability 1 when following conditions are satisfied:

$$\sum_{n=1}^{\infty} \alpha_n(a) = \infty \text{ and } \sum_{n=1}^{\infty} \alpha_n^2(a) < \infty$$

But these varying step size is seldom used in applications and empirical search, since it often converge slowly or need considerable tuning in order to obtain a satisfactory convergence rate.

1.3.1.2 Selection of Actions

1.3.1.2.1 Greedy Action Selection The simplest way of action selection is to select the action with the largest value, which is called the *greedy action*.

Algorithm 3: Greedy Action Selection Algorithm

Input : $\mathcal{A}$ set of all possible actions
 t current time step
 $Q_t(a)$ estimated value of action a at time step t

Output: A_t action selection at time t

1 $A_t \leftarrow \arg \max_a Q_t(a)$
2 **return** A_t

1.3.1.2.2 ϵ -greedy Selection Since what we can know is only the estimate of the value in current time step, the greedy action may not be the best action choice in the long run. Selecting the action other than greedy action is called **exploring**. One of the simple method of exploration is to select non-greedy actions with certain probability ϵ .

Algorithm 4: ε -greedy Selection

Input : $\mathcal{A}$ set of all possible actions
 t time step
 $Q_t(a)$ estimation of value of action a on time step t
 $\varepsilon \in [0, 1]$ probability

Output: A_t action selection at time t

```
1  $p \sim \mathcal{U}(0, 1)$ 
2 if  $p > \varepsilon$  then
3   |  $A_t \leftarrow \arg \max_a Q_t(a)$ 
4 else
5   |  $A_t \sim \mathcal{A}$ 
6 return  $A_t$ 
```

With probability $1 - \varepsilon$, the algorithm exploits. However, with probability ε , the algorithm explores. Note that the exploration does not exclude the greedy action.

1.3.1.2.3 Upper-Confidence-Bound Action Selection UCB action selection takes into account both how close the estimates of action values are to being maximal and the uncertainties in those estimates.

Algorithm 5: Upper-Confidence-Bound Action Selection

Input : $\mathcal{A}$ set of all possible actions
 t time step
 $Q_t(a)$ estimation of value of action a on time step t
 $N_t(a)$ the number of times action a was selected prior to time t
 c degree of exploration

Output: A_t action selection at time t

```
1  $A_t \leftarrow \arg \max_a \left\{ Q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}} \right\}$ 
2 return  $A_t$ 
```

The square root term can be seen as uncertainty term. Each time a is selected, the uncertainty term decreases. On the other hand, when a is not selected for a long time, the uncertainty increases.

The UCB action selection method cannot be extended to general non-stationary bandit problems, and has difficulty dealing with large state spaces.

1.3.1.3 Optimistic Initial Values

By simply setting the initial values $Q_0(\cdot)$ optimistically, we can encourage exploration easily. First, the algorithm sets initial values equally with high, optimistic values. Then the algorithm selects action, like normal methods. Since the values were highly optimistic, the rewards that the algorithm receives will likely be disappointing, encouraging other actions to be selected next time. The result is that all actions are tried several times before the value estimates converge.

This method can outperform ε -greedy method with realistic initial values when combined with greedy action selection method. However, it cannot be used in non-stationary problems, since its drive for exploration is effective only in the beginning steps.

1.3.2 Gradient Bandit Algorithm

Instead of estimating the action values and deciding which action to execute depending on the estimates, we can learn the *preference* of actions and use it to maximize the expected reward. This preference has no interpretation in terms of rewards; the higher the preference, the more often the action is selected. The preference is not absolute; it is relative values among the actions. If the preferences of actions are same, for example, 1000, the probability of each action being selected equals each other. We denote the preferences of action a in time step t as $H_t(a)$. The probabilities of choosing the action a at time step t is usually calculated using the *softmax* function, given as:

$$\mathbb{P}\{A_t = a\} = \frac{e^{H_t(a)}}{\sum_{b \in \mathcal{A}} e^{H_t(b)}}, \quad (1.1)$$

and we denote it as $\pi_t(a)$.

1.3.2.1 Algorithm

Algorithm 6: Gradient Bandit Algorithm

Input : $\mathcal{A}$ set of all possible actions
 $\{H_0(a)\}_{a \in \mathcal{A}}$ initial preferences
 α step size

Output: $\{H(a)\}$ final preferences

```

1  $t \leftarrow 0$ 
2  $\bar{R}_t \leftarrow 0$ 
3 repeat
4   foreach  $a \in \mathcal{A}$  do
5      $\pi_t(a) \leftarrow \frac{e^{H_t(a)}}{\sum_{b \in \mathcal{A}} e^{H_t(b)}}$ 
6    $A_t \sim \text{Categorical}(\{\pi_t(a)\}_{a \in \mathcal{A}})$ 
7    $R_t \leftarrow$  the given reward
8    $\bar{R}_t \leftarrow$  the average (or weighted average for nonstationary) of rewards up through and including time  $t$ 
9   foreach  $a \in \mathcal{A}$  do
10     $H_{t+1}(a) \leftarrow H_t(a) + \alpha(R_t - \bar{R}_t)(\mathbb{1}_{a=A_t} - \pi_t(a))$ 
11   $t \leftarrow t + 1$ 
12 until convergence
13 return  $\{H_t(a)\}_{a \in \mathcal{A}}$ 

```

Remark 1.3.2.1 (Gradient Bandit Algorithm as Stochastic Gradient Ascent)

The gradient bandit algorithm can be derived using stochastic approximation to gradient ascent. First of all, our objective is to maximize the expected reward:

$$\mathbb{E}[R_t] = \sum_{a \in \mathcal{A}} \pi_t(a) q_*(a) \quad (1.2)$$

where $\pi_t(a)$ is given as 1.1. The update equation of exact gradient ascending would then be

$$H_{t+1}(a) = H_t(a) + \alpha \frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)} \quad (1.3)$$

Taking a closer look at the update equation, we can get the following result;

$$\frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)} = \frac{\partial}{\partial H_t(a)} \left\{ \sum_{x \in \mathcal{A}} \pi_t(x) q_*(x) \right\} \quad (1.4)$$

$$= \sum_x q_*(x) \frac{\partial \pi_t(x)}{\partial H_t(a)} \quad (1.5)$$

$$= \sum_x (q_*(x) - B_t) \frac{\partial \pi_t(x)}{\partial H_t(a)} \quad (1.6)$$

where B_t is called *baseline*, and can be any value that does not depend on x . Note that the sum of probability is always 1, and this enables the equality of 1.6. Next we multiply equation by $1 = \frac{\pi_t(x)}{\pi_t(x)}$;

$$\frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)} = \sum_x \pi_t(x) (q_*(x) - B_t) \frac{\partial \pi_t(x)}{\partial H_t(a)} / \pi_t(x) \quad (1.7)$$

$$= \mathbb{E} \left[(q_*(A_t) - B_t) \frac{\partial \pi_t(A_t)}{\partial H_t(a)} / \pi_t(A_t) \right] \quad (1.8)$$

$$= \mathbb{E} \left[(R_t - \bar{R}_t) \frac{\partial \pi_t(A_t)}{\partial H_t(a)} / \pi_t(A_t) \right] \quad (1.9)$$

where we have chosen the baseline $B_t = \bar{R}_t$ and substituted $q_*(A_t) = R_t$. Note that

$$\frac{\partial \pi_t(x)}{H_t(a)} = \frac{\partial}{\partial H_t(a)} \pi_t(x) \quad (1.10)$$

$$= \frac{\partial}{\partial H_t(a)} \left[\frac{e^{H_t(x)}}{\sum_{b \in \mathcal{A}} e^{H_t(b)}} \right] \quad (1.11)$$

$$= \frac{\frac{\partial e^{H_t(x)}}{\partial H_t(a)} \sum_{b \in \mathcal{A}} e^{H_t(b)} - e^{H_t(x)} \frac{\partial \sum_{b \in \mathcal{A}} e^{H_t(b)}}{\partial H_t(a)}}{\left(\sum_{b \in \mathcal{A}} e^{H_t(b)} \right)^2} \quad (1.12)$$

$$= \frac{\mathbb{1}_{a=x} e^{H_t(x)} \sum_{b \in \mathcal{A}} e^{H_t(b)} - e^{H_t(x)} e^{H_t(a)}}{\left(\sum_{b \in \mathcal{A}} e^{H_t(b)} \right)^2} \quad (1.13)$$

$$= \frac{\mathbb{1}_{a=x} e^{H_t(x)}}{\sum_{b \in \mathcal{A}} e^{H_t(b)}} - \frac{e^{H_t(x)} e^{H_t(a)}}{\left(\sum_{b \in \mathcal{A}} e^{H_t(b)} \right)^2} \quad (1.14)$$

$$= \mathbb{1}_{a=x} \pi_t(x) - \pi_t(x) \pi_t(a) \quad (1.15)$$

$$= \pi_t(x) (\mathbb{1}_{a=x} - \pi_t(a)). \quad (1.16)$$

Substituting this result to 1.9, we get

$$\frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)} = \mathbb{E} [(R_t - \bar{R}_t) \pi_t(A_t) (\mathbb{1}_{a=A_t} - \pi_t(a)) / \pi_t(A_t)] \quad (1.17)$$

$$= \mathbb{E} [(R_t - \bar{R}_t) (\mathbb{1}_{a=A_t} - \pi_t(a))]. \quad (1.18)$$

Evaluating this expectation by sampling and substituting to 1.3 leads to

$$H_{t+1}(a) = H_t(a) + \alpha (R_t - \bar{R}_t) (\mathbb{1}_{a=A_t} - \pi_t(a)) \quad (1.19)$$

Chapter 2

Markov Decision Processes

2.1 Introduction

In this chapter, we discuss **Markov decision process**, a framework that reinforcement learning heavily relies on. Markov decision process is the foundation of reinforcement learning algorithms. We mainly focus on discrete Markov decision process.

The following property of conditional expectation is used often in this document; for example in the derivation of Bellman equations.

$$\mathbb{E}[\mathbb{E}[X|Y, Z]|Y] = \mathbb{E}[X|Z] \quad (2.1)$$

2.2 Finite Markov Decision Process

2.2.1 Definition

The learner and decision maker is called *agent*, and the *environment* is the thing the agent interacts with. These two interact at each of a sequence of time step $t = 0, 1, 2, 3, \dots$. At every time step t , the agent receives *state* $S_t \in \mathcal{S}$ of the environment and selects an action $A_t \in \mathcal{A}(s)$ based on that state s . Next step, the agent receives a reward $R_{t+1} \in \mathcal{R}$ and finds itself in a new state S_{t+1} . The MDP then give rise to the *trajectory* $S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$.

The states, rewards and actions are all have finite number of elements in *finite* MDP. We will mainly discuss about finite MDP.

Definition 2.2.1.1 (Markov Decision Process (MDP))

Markov decision process is a tuple of five objects $(\mathcal{S}, \mathcal{A}, p, \mathcal{R}, \gamma)$ where $\mathcal{S}$, $\mathcal{R}$, and $\mathcal{A}$ are sets of states, rewards, and actions. The function $p : \mathcal{S} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ is called *state-transition probability* and defined as

$$p(s'|s, a) = \mathbb{P}\{S_t = s' | S_{t-1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r | s, a).$$

The function $p(s', r | s, a) = \mathbb{P}\{S_t = s', R_t = r | S_{t-1} = s, A_{t-1} = a\}$ is called *dynamics* of the MDP. $\gamma \in [0, 1]$ is called *discount rate*, and is used to calculate the expected value of cumulative rewards, namely the *return* and the expectation of it is what the agent has to maximize. The return is defined as

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

Note that the goal can be written in recursive manner; $G_t = R_{t+1} + \gamma G_{t+1}$.

2.2.2 Markov Property

The important property of Markov decision process is that R_t and S_t depends only on previous state and action; S_{t-1} and A_{t-1} . This is called **Markov property**. More formally, the Markov property can be defined as following;

Definition 2.2.2.1 (Markov Property)

Let S_t , R_t and A_t be the state, reward and action at time t . Then, the probability distribution satisfies **Markov property** if

$$P\{S_t = s', R_t = r | S_{t-1} = s_{t-1}, A_{t-1} = a_{t-1}, \dots, S_0 = s_0, A_0 = a_0\} = P\{S_t = s', R_t = r | S_{t-1} = s_{t-1}, A_{t-1} = a_{t-1}\}$$

for all $s', s_i \in \mathcal{S}$, $r \in \mathcal{R}$ and $a_i \in \mathcal{A}$.

2.3 Episodic Task and Continuing Task

The *return* $G_t = R_{t+1} + \gamma^2 R_{t+2} + R_{t+3} + \dots$, presented in Definition 2.2.1.1 shortly, is the value which we should maximize about expectation. The return may consist of finite rewards, or it can be formulated as an infinite sum.

When the agent-environment interaction can break naturally into subsequences, for example winning the chess game and starting it all again from the start, we call these subsequences **episodes**. All the episodes terminate in the special state called **terminal state**. Then, the return can be expressed in finite sum;

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-1} R_T$$

where the time of termination, T is a random variable that normally varies from episode to episode and $0 \leq \gamma \leq 1$ is the discount rate. The states other than terminal state is often written as $\mathcal{S}$, and all the states including terminal state is often written as $\mathcal{S}^+$.

When the agent-environment interaction cannot be expressed in terms of episodes, we call it **continuing task**. The

return now cannot be formulated as finite sum;

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.2)$$

$$= R_{t+1} + \gamma G_{t+1} \quad (2.3)$$

This equation still holds when the task is episodic; We can set $G_T = 0$ for terminal time T .

The episodic and continuing tasks can be expressed in unified notation by using special *absorbing state* which transitions only to itself and generates only zero rewards. Then the return can be expressed as

$$G_t = \sum_{k=t+1}^T \gamma^{k-t-1} R_k$$

with possibility $T = \infty$ and $\gamma = 0$, but not both.

2.4 Policies and Value Functions

Now we introduce *value functions* which represents the estimate of *how good* it is for the agent to be in a given state or to perform a given action in a given state. The 'how good' is expressed in terms of expectations of returns.

2.4.1 Definitions

Definition 2.4.1.1 (Policy)

The **policy** $\pi(a|s)$ is a mapping from states to probabilities of selecting each possible action, where $a \in \mathcal{A}(s)$.

If the agent is following policy $\pi(a|s)$ at time t , the probability of selecting $A_t = a$ in $S_t = s$ is $\pi(a|s)$.

Definition 2.4.1.2 (Value Function)

The **value function** $v_{\pi}(s)$ of a state s is the expected return when starting in s and following policy π , i.e.,

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s \right]$$

where $\mathbb{E}_{\pi}$ denotes the expected value of a random variable given that the agent follows policy π . The function v_{π} itself is called **state-value function for policy π** .

The value of terminal state is always 0.

Definition 2.4.1.3 (Action-Value Function)

The **action-value function** $q_{\pi}(s, a)$ is the expected return starting from state s , taking the action a , and then following policy π .

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s, A_t = a \right]$$

Note that the value function and action-value function has following relationship:

Theorem 2.4.1.1 (Relationship of Value Function and Action-Value Function)

The state-value function $v_\pi(s)$ and action-value function $q_\pi(s, a)$ satisfies following property:

$$v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a|s) q_\pi(s, a)$$

$$q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s, A_t = a]$$

Proof. Use the equation 2.1. The second equation can be derived using equation 2.1 with the Markov property. Note that the expectation is taken about policies. $\square$

These functions can be estimated from experience. This type of estimation method is one of Monte Carlo methods. When there are too much states and actions or when they are continuous, we can approximated the function using parametrized approximator, such as deep neural network.

2.4.2 Bellman Equation

The fundamental property of value functions that are used throughout reinforcement learning and dynamic programming is that they satisfy recursive relationship, called **Bellman equation** for v_π .

Theorem 2.4.2.1 (Bellman Expectation Equation for v_π)

Let v_π be a state-value function for policy π . Then the v_π satisfies following equation;

$$v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] \tag{2.4}$$

$$= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s] \tag{2.5}$$

$$= \sum_{a \in \mathcal{A}(s)} \pi(a|s) \sum_{s'} \sum_r p(s', r | s, a) [r + \gamma \mathbb{E}_\pi[G_{t+1} | S_{t+1} = s']] \tag{2.6}$$

$$= \sum_{a \in \mathcal{A}(s)} \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')], \text{ for all } s \in \mathcal{S} \tag{2.7}$$

which is called the **Bellman equation** for v_π .

The actions a are taken from $\mathcal{A}(s)$. The sum can be understood as expectation, since it is a sum over all three variables a , s , and r . It is just a sum of returns $r + \gamma v_\pi(s')$ multiplied by the probability $\pi(a|s)p(s', r | s, a)$.

The following is the Bellman equation for action-value function.

Theorem 2.4.2.2 (Bellman Expectation Equation for q_π)

Let q_π be an action-value function for policy π . Then q_π satisfies following equation;

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a] \quad (2.8)$$

$$= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a] \quad (2.9)$$

$$= \sum_{s', r} p(s', r | s, a) [r + \gamma \mathbb{E}_\pi[G_{t+1} | S_{t+1} = s']] \quad (2.10)$$

$$= \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')] \quad (2.11)$$

$$= \sum_{s', r} p(s', r | s, a) \left[r + \gamma \sum_{a' \in \mathcal{A}(s')} \pi(a' | s') q_\pi(s', a') \right], \text{ for all } s \in \mathcal{S}, a \in \mathcal{A}(s) \quad (2.12)$$

which is called the **Bellman equation** for q_π .

Note that we used law of total expectation; check 2.1.

2.5 Optimal Policies and Optimal Value Functions

2.5.1 Definition

Value functions define a partial ordering over policies. We denote $\pi \geq \pi'$ if and only if $v_\pi(s) \geq v_{\pi'}(s)$ for all $s \in \mathcal{S}$. The **optimal policy** is a policy that is better than or equal to all other policies. There is always at least one optimal policy. There can be more than one optimal policies, but they all share the same state-value function, namely the *optimal state-value function* and the same *optimal action-value function*.

Definition 2.5.1.1 (Optimal State-Value Function)

Let v_π be a state-value function. Then, the **optimal state-value function** v_* is defined as

$$v_*(s) = \max_{\pi} v_\pi(s)$$

for all $s \in \mathcal{S}$.

Definition 2.5.1.2 (Optimal Action-Value Function)

Let q_π be an action-value function. Then, the **optimal action-value function** q_* is defined as

$$q_*(a, s) = \max_{\pi} q_\pi(a, s)$$

for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$.

2.5.2 Bellman Optimality Equation

The optimal functions presented above also satisfies the Bellman equation, called the **Bellman optimality equation**;

Theorem 2.5.2.1 (Bellman Optimality Equation)

Let v_* and q_* be optimal state-value function and optimal action-value function. Then, these functions satisfy the following **Bellman optimality equation**;

$$v_*(s) = \max_{a \in \mathcal{A}} q_*(s, a) \quad (2.13)$$

$$= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a] \quad (2.14)$$

$$= \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')] \quad (2.15)$$

$$q_*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') | S_t = s, A_t = a] \quad (2.16)$$

$$= \sum_{s', r} p(s', r | s, a) [r + \gamma \max_{a'} q_*(s', a')] \quad (2.17)$$

Note that we used following relationship between optimal functions;

Theorem 2.5.2.2 (Relationship Between Optimal Functions)

Let v_* and q_* be optimal state-value function and optimal action-value function. Then, these functions satisfy the following relationship;

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s, a) \quad (2.18)$$

Moreover, following holds for optimal policy π_* ;

$$\pi_*(s) = \arg \max_{a \in \mathcal{A}(s)} q_*(s, a) \quad (2.19)$$

Proof. The proof for first equation uses $a = b \Leftrightarrow a \leq b$ and $b \leq a$. The second equation can be proved using the first equation. $\square$

Once one has v_* , determining an optimal policy is straightforward. We just have to make one step along, and then find out the actions that satisfy the Bellman optimality equation. We then create the policy that allocates positive probability only to these actions. This *greedy* selection seems that it ignores possibility of better alternatives in the long run. However, note that we are trying to maximize the *return*, which considers a long-run cumulative sum, and the optimal functions are derived using this return. Thus, we do not have to worry about the better alternatives.

Having q_* , it is even more straightforward to create an optimal policy. We don't even have to take a step ahead; since we can always find optimal action a , for any given state s .

Chapter 3

Dynamic Programming

3.1 Introduction

Dynamic programming (DP) refers to a collection of algorithms which finds out optimal policies given a *perfect* model of MDP. Since it requires perfect MDP model and has expensive computational cost, DP is not used so much in real applications. However, the methods shown later (Monte Carlo, Temporal Difference) are all rooted from dynamic programming. DP is a foundation of these methods.

3.2 Policy Evaluation

First, we consider how to compute the state-value function v_π when a policy π is given. Using the Bellman equation, we can get an *iterative policy evaluation algorithm*. Starting with initial state-value function v_0 , we update the state-value function as following;

$$v_{k+1}(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_k(S_{t+1} | S_t = s)] \quad (3.1)$$

$$= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_k(s')] \quad (3.2)$$

for all $s \in \mathcal{S}$. We can see that $v_k = v_\pi$ is the fixed point for this update rule. The sequence $\{v_k\}$ is known to converge to v_π under the same condition that guarantee the existence of v_π . The algorithm pseudocode is given as follows.

Algorithm 7: Iterative Policy Evaluation Algorithm

Input : $p(s', r|s, a)$ the dynamics of given finite MDP
 π given policy
Output: v_π the state-value function

```
1  $\theta \leftarrow$  appropriate threshold value for convergence
2 initialize  $V(s)$  for all  $s \in \mathcal{S}^+$  except that  $V(\text{terminal}) = 0$ 
3 repeat
4    $\Delta \leftarrow 0$ 
5   foreach  $s \in \mathcal{S}$  do
6      $v \leftarrow V(s)$ 
7      $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a)[r + \gamma V(s')]$ 
8      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
9 until  $\Delta < \theta$ 
10 return  $V \simeq v_\pi$ 
```

One can notice that the algorithm 7 is not really same as the update equation 3.2. When updating the value of state, we may use the values of states that are newly updated, unlike the equation 3.2. However, this is also known to converge well, even more quickly than when only using the old values of states. This is intuitively reasonable since the new values can be regarded more closer to the real value.

3.3 Policy Improvement

Now we have evaluated the value function. Our next step is to improve the policy with the state-value function we computed. The improvement of the policy has its root on the *policy improvement theorem*:

Theorem 3.3.0.1 (Policy Improvement Theorem)

Let π , v_π and q_π be a given deterministic policy and state-value function and action-value function of the policy π , respectively. Let π' be another deterministic policy. Then, the following holds:

$$q_\pi(s, \pi'(s)) \geq v_\pi(s) \Rightarrow v_{\pi'}(s) \geq v_\pi(s) \text{ for all } s \in \mathcal{S} \quad (3.3)$$

Moreover, if there is a strict inequality of the equation 3.3 of left-hand side at any state, then there must be strict inequality of the right-hand side of the equation 3.3 at that state.

The idea behind the proof of this statement is quite straightforward. We let the policy π' to be identical to π except that

$\pi'(s) = a \neq \pi(s)$. It is given as follows:

$$\begin{aligned}
v_\pi(s) &\leq q_\pi(s, \pi'(s)) \\
&= \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s, A_t = \pi'(s)] \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s] \\
&\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) | S_t = s] \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma E_{\pi'}[R_{t+2} + \gamma v_\pi(S_{t+2}) | S_{t+1}, A_{t+1} = \pi'(S_{t+1})] | S_t = s] \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2}) | S_t = s] \\
&\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 v_\pi(S_{t+3}) | S_t = s] \\
&\vdots \\
&\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots | S_t = s] \\
&= v_{\pi'}(s).
\end{aligned}$$

Now, considering the changes at all states and to all possible actions, we consider the new *greedy* policy π' given by

$$\pi'(s) = \arg \max_{a \in \mathcal{A}(s)} q_\pi(s, a) \quad (3.4)$$

$$= \arg \max_{a \in \mathcal{A}(s)} \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s, A_t = a] \quad (3.5)$$

$$= \arg \max_{a \in \mathcal{A}(s)} \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')] \quad (3.6)$$

This process, making the original policy greedy with respect to the value function of the original policy, is called *policy improvement*.

It is guaranteed for a policy to strictly improve unless the original policy is already optimal; Let π' be a policy that is as good but not better than π . Then, $v_\pi = v_{\pi'}$ and from the equation 3.5, it follows that for all $s \in \mathcal{S}$,

$$\begin{aligned}
v_{\pi'}(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v_{\pi'}(S_{t+1}) | S_t = s, A_t = a] \\
&= \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi'}(s')].
\end{aligned}$$

And this is same as the Bellman optimal equation, and $v_{\pi'}$ must be v_* , the optimal value function, and so is v_π .

3.4 Obtaining Optimal Policy

3.4.1 Policy Iteration

Now, we can repeat the above two processes, policy evaluation and policy improvement, to obtain the optimal policy and optimal state-value function. Specifically, we start with initial policy π_0 and evaluate the state-value function v_{π_0} . Then we improve policy π_0 with v_{π_0} to get π_1 , and evaluate next state-value function v_{π_1} . And we repeat it until we get π_* and v_* .

This way of finding optimal policy is called *policy iteration*.

Since the finite MDP has only finite number of policies, this process must converge to an optimal policy in finite number of iterations.

Algorithm 8: Policy Iteration (with iterative policy evaluation)

Input : $p(s', r|s, a)$ the dynamics of given finite MDP
 π initial policy
 v initial state-value function
Output: π_* optimal policy
 v_* optimal state-value function

```
1  $\theta \leftarrow$  appropriate threshold value for convergence
2  $\Delta \leftarrow 0$ 
3 policy-stable  $\leftarrow$  false
4 repeat
    // Policy Evaluation
5   repeat
6      $\Delta \leftarrow 0$ 
7      $V_{old}(s) \leftarrow V(s)$ 
8     foreach  $s \in \mathcal{S}$  do
9        $v \leftarrow V(s)$ 
10       $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a)[r + \gamma V(s')]$ 
11       $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
12   until  $\Delta < \theta$ 
    // Policy Improvement
13   policy-stable  $\leftarrow$  true
14   foreach  $s \in \mathcal{S}$  do
15      $\pi'(s) \leftarrow \pi(s)$ 
16      $\pi(s) \leftarrow \arg \max_{a \in \mathcal{A}(s)} \sum_{s', r} p(s', r|s, a)[r + \gamma V(s')]$ 
17     if  $\pi'(s) \neq \pi(s)$  then
18       policy-stable  $\leftarrow$  false
19 until policy-stable is true and  $\max |V_{old}(s) - V(s)| < \theta$ 
```

3.4.2 Value Iteration

One drawback of policy iteration is that it involves policy evaluation, which may take a long time if the state space is too large. The good news is that the policy evaluation step in policy iteration can be truncated in several ways, without losing a convergence guarantees of policy iteration. One of such a way is *value iteration*. The value iteration combines policy improvement and truncated policy evaluation steps;

$$v_{k+1} = \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s, A_t = a] \quad (3.7)$$

$$= \max_a \sum_{s', r} p(s', r|s, a)[r + \gamma v_k(s')] \quad (3.8)$$

The sequence $\{v_k\}$ is known to converge to v_* .

Algorithm 9: Value Iteration

Input : $p(s', r|s, a)$ the dynamics of given finite MDP
 $V(s)$ initial value function, except that $V(\text{terminal}) = 0$
Output: π final policy

```
1  $\theta \leftarrow$  appropriate threshold value
2 repeat
3    $\Delta \leftarrow 0$ 
4   foreach  $s \in \mathcal{S}$  do
5      $v \leftarrow V(s)$ 
6      $V(s) \leftarrow \max_a \sum_{s', r} p(s', r|s, a)[r + \gamma V(s')]$ 
7      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
8 until  $\Delta < \theta$ 
9 foreach  $s \in \mathcal{S}$  do
10   $\pi(s) \leftarrow \arg \max_a \sum_{s', r} p(s', r|s, a)[r + \gamma V(s')]$ 
11 return  $\pi$ 
```

Faster convergence can often be achieved by interposing multiple policy evaluation sweeps between each policy improvement step.

3.5 Asynchronous Dynamic Programming

As one can see, the DP methods requires a sweep along the state space for policy evaluation. This can be expensive, if the given state space is too large.

Asynchronous DP algorithms do not sweep along the state space. The values are updated without any order in this algorithms, and even some states can be updated multiple times while other states are not. However, it cannot ignore the states; for the asynchronous DP algorithms to converge, they must visit all states.

The point is that we do not have to wait for policy evaluation step to finish. We can improve our policy without sweeping a state space for policy evaluation. We might even try to skip updating some states if it is irrelevant to optimal behavior.

Asynchronous DP algorithms can easily be mixed up with real-time interaction.

3.6 Generalized Policy Iteration

Policy iteration involves two simultaneous interacting processes, the policy evaluation and policy improvement. In policy iteration method, these two processes wait each other to be finished. But, this is not necessary, and the example of it is the value iteration. In asynchronous DP algorithms, these two processes intervene in a much finer manner. We use the term **general policy iteration** to denote the general idea of letting policy evaluation and improvement processes interact.